

Lab 1: Requirements Engineering & UML Use-Case Modelling

Problem Statement #02: Automated Rubric Assignment Evaluator

1. Requirements Specification Table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall automatically queue uploaded student project archive files (.zip/.tar.gz), execute pre-configured unit test suites, and generate an itemized rubric score breakdown.	High	Pass: Rubric breakdown is rendered with unit test results within 10 seconds of processing. Fail: Queue stalls, unhandled execution exception occurs, or score breakdown fails to generate.	Core evaluation workflow that automates baseline code assessment and scoring against test suites.
FR-002	Functional	The system shall perform static syntax validation and dependency verification prior to test execution, halting execution if unauthorized packages or syntax errors are detected.	High	Pass: System flags syntax errors and prohibited packages, logging diagnostic feedback before execution. Fail: Invalid code crashes the runner or executes with blacklisted packages.	Prevents execution of broken or malicious code and ensures adherence to language constraints.
FR-003	Functional	The system shall allow Faculty Evaluators to define weighted rubric criteria (e.g., test cases, style, documentation) and manually adjust automated scores with qualitative remarks.	High	Pass: Criteria weights validate to 100%, and score adjustments recalculate totals with an audit trail. Fail: Invalid weights are accepted or overrides fail to update the final score.	Provides faculty with necessary oversight and flexibility to adjust grades based on qualitative factors.
FR-004	Functional	The system shall automatically distribute anonymized submissions to designated student peers for peer review according to predefined allocation rules upon deadline closure.	Medium	Pass: Every eligible submission is allocated to the configured number of peers (e.g., 2 peers) with all student identities redacted. Fail: Author identity is exposed or submissions remain unassigned.	Eliminates manual distribution overhead while ensuring unbiased and fair peer evaluations.
FR-005	Functional	The system shall allow students to view their itemized rubric scores, execution logs, and peer feedback, and submit a regrade request within an active dispute window.	Medium	Pass: Students can view their own itemized breakdown and successfully file a regrade request with a mandatory rationale. Fail: Unauthorized access to peer submissions occurs or regrade requests are permitted after window expiry.	Ensures grading transparency and provides a structured mechanism for student feedback and appeals.
NFR-001	Non-Functional (Performance & Scalability)	The system shall handle up to 100 concurrent project submissions without dropping queue items or exceeding a 1.5 GB memory footprint.	High	Pass: Load tests confirm zero queue loss, memory usage $\leq$ 1.5 GB, and average response latency $\leq$ 10 seconds under 100 concurrent requests. Fail: Server drops requests, crashes, or memory usage exceeds 1.5 GB.	Guarantees system reliability and responsiveness during peak submission deadlines.

NFR-002	Non-Functional (Security & Isolation)	The system shall execute student code within isolated sandbox containers with network access disabled and a strict 15-second execution timeout.	High	Pass: Sandbox enforces execution limits, terminates infinite loops after 15 seconds, and blocks outbound network traffic. Fail: User code accesses host filesystem, external network, or runs beyond 15 seconds.	Protects server infrastructure and grading integrity against malicious scripts and runaway processes.
---------	---	---	------	---	---